

Diagnostics

Overview

The diagnostic module is module ID 7. It coordinates active deployment, hardware, integration, and stress tests and is loaded when a diagnostic is first requested.

Older messages and configuration material can use the name NVIDIA Validation Suite (NVVS). The supported customer workflow runs DCGM Diagnostics through `dcgmi diag`; this chapter uses *diagnostic* for that functionality.

This chapter explains when to use diagnostics, how the available tests fit together, and how to prepare and interpret a run. For exhaustive invocation syntax, canonical names, parameters, output, and exit status, see [dcgmi diag](#). Detailed behavior for each test is in [Diagnostic Plugins](#).

When diagnostics are useful

Diagnostics are appropriate when the question requires active evidence that the accelerator and its surrounding software or interconnect can execute a known test. Supported suites primarily target GPUs and can also include specific NVIDIA CPU diagnostics when the required hardware and plugin are present. Typical points in an operational workflow are:

- a quick readiness test before a node accepts work;
- a medium validation after a failed job, once the GPUs are idle;
- a selected PCIe, memory, compute, power, or bandwidth test while isolating a suspected fault; or
- a long administrator-led validation before returning a node to service.

Diagnostics differ from [Health Monitoring](#), which evaluates telemetry collected during ordinary operation without applying a test workload. A node can have no passive health incidents and still fail an active diagnostic, or can retain a historical health warning after the underlying hardware passes a later test. Use both results as evidence in the wider node and workload workflow.

Warning

Diagnostics can consume GPU compute, memory, PCIe, NVLink, CPU, power, and cooling capacity. Longer suites can reset state or conflict with running processes. Select explicit entities, coordinate with the scheduler, and run invasive tests only on resources that can be taken out of service.

Goals

DCGM Diagnostics are designed to:

- assess node readiness before a workload is deployed;
- return human-readable or structured results for interactive and automated workflows;
- offer quick, medium, long, and extra-long timeframes for prologue, failed-job, and administrator-led investigation;
- identify deployment and software problems involving NVML, CUDA, device access, conflicting software, and platform services;
- exercise memory, compute, PCIe, NVLink, power, thermal, and collective- communication paths; and
- provide error codes, categories, and severity information that a cluster manager can use when deciding whether to monitor, reset, triage, or isolate a resource.

Beyond Scope

DCGM Diagnostics do not repair faults, replace NVIDIA offline field diagnostics, or determine RMA eligibility. They run inside the installed software and platform environment, so a failure can originate in the host, permissions, libraries, topology, cooling, power, or workload interference as well as in the target processor.

Run Levels and Tests

The predefined suites increase in scope and runtime. Use the quick suite as a readiness check, the medium suite when investigating a failed workload, and the long or extra-long suites for administrator-led isolation and post-mortem testing. Later suites include the checks from earlier suites and add more invasive hardware, interconnect, performance, and stress tests.

Suite composition and individual-test availability depend on the DCGM release, loaded diagnostic plugins, target entities, hardware, libraries, executables, and privileges. See [DIAGNOSTIC TESTS](#) for suite composition, accepted suite selectors, documented test names, and availability notes.

Note

DCGM supports diagnostics on datacenter and non-datacenter GPUs, including GeForce and Quadro GPUs. Non-datacenter GPUs support level 1; higher levels are available only where support is explicitly documented for the product. See [Feature Availability by GPU Class](#).

Diagnostic plugins

The following table is a selection guide, not the exhaustive test reference. The link on each canonical diagnostic name opens its detailed page under Reference.

Diagnostic	Defects or conditions it is designed to identify
software	Missing or unusable NVML/CUDA libraries; inaccessible device nodes or cgroup restrictions; conflicting drivers or processes; problematic environment variables; page-retirement, row-remap, InfoROM, memory-health, or Fabric Manager state. Run level 1 to select these built-in checks alone.
context_create	Failure to resolve a GPU and create a basic CUDA context because of driver initialization, device access, or incompatible compute mode.
memory	GPU framebuffer allocation, write/read, data-integrity, ECC, and L1 cache/tag faults.
pcie	Low host/device or peer bandwidth, excessive latency, broken peer access, PCIe replays or AER events, and NVLink state or error problems.
diagnostic	Compute or framebuffer data-path errors under sustained matrix work, including mismatches, XIDs, ECC events, thermal excursions, and unexpectedly inconsistent performance.
memory_bandwidth	Local framebuffer bandwidth below the device threshold or inability to complete the bandwidth workload.
targeted_stress	Inability to sustain a requested compute rate, unstable performance, or telemetry errors while a controlled GEMM workload runs.
targeted_power	Inability to reach the requested power threshold or errors observed under sustained power load.
nvbandwidth	Failed GPU copy paths over NVLink or PCIe and unavailable or misconfigured peer paths.
memtest	GPU memory-cell, address-line, stuck-bit, coupling, retention, and data-pattern errors across a configurable set of memory tests.
pulse_test	Board or power-delivery instability exposed by rapid changes in GPU current and power demand.

Preparing a Run

Command-line invocation

Use `dcgmi diag` to select a numbered suite or canonical named tests, target entities, override test parameters, repeat a run, and choose text or JSON output. An explicitly named test also performs the built-in software checks. The command reference documents selection grammar and interactions between options.

Before an invasive run:

- discover and select explicit entities;
- drain applications and peer users from those entities;
- confirm that the diagnostic service account can access the device nodes, required libraries, plugin files, and external test executables; and
- choose a timeout longer than the requested test durations.

Environment

The host-engine environment controls diagnostic child execution:

Diagnostic environment variables

Variable	Purpose
<code>NVVS_BIN_PATH</code>	Directory in which the diagnostic module searches for the <code>nvvs</code> executable.
<code>NVVS_PLUGIN_DIR</code>	Readable and traversable directory in which <code>nvvs</code> searches for diagnostic plugins.
<code>NVVS_HANGDETECT_DISABLE</code>	Disable diagnostic hang detection. Hang detection is enabled by default.
<code>NVVS_HANGDETECT_EXPIRY_SEC</code>	Inactivity timeout for diagnostic threads or processes. The value must be at least 120 seconds and divisible by 60.

These are read by the host engine and diagnostic processes, not retroactively from the shell that invokes `dcgmi`.

Exit status

For `dcgmi` shell-status conversion and diagnostic-specific failure statuses, see [EXIT STATUS](#). A nonzero status identifies a request, launch, connection, or diagnostic error. Automation should also parse the diagnostic results: test status, warnings, skipped tests, and per-entity errors carry information that a single process status cannot express.

Results and Logging

Text output is useful interactively. It begins with run metadata, then groups the tests returned by the diagnostic into categories. This illustrative single-GPU excerpt shows the result hierarchy for a failed test:

```
Successfully ran diagnostic for group.
+-----+-----+
| Diagnostic | Result |
+-----+-----+
|---- Metadata ----+-----+
| DCGM Version | <installed-version> |
| Driver Version Detected | <driver-version> |
| GPU Device IDs Detected | <device-id> |
+---- Hardware ----+-----+
| memory | Fail |
| | GPU0: Fail |
| Warning: GPU0 | <condition-specific error message> |
+-----+-----+
```

The named-test row is the test summary, the indented entity row is that entity’s result, and the warning row supplies entity-specific context. Exact metadata, categories, tests, statuses, and messages vary with the release, request, installed components, target, and observed result.

JSON preserves test names, per-entity statuses, warnings, error codes, categories, severities, and auxiliary data for release-specific automation. It is not governed by a stable schema; see [JSON output](#) before making it an integration boundary. See [Diagnostic Errors](#) for the error taxonomy and response guidance.

`dcgmi diag` can also control the server-side encrypted diagnostic debug log and plugin statistics files. Retain the command, target selection, complete output, debug log, and relevant plugin statistics when escalating a failure. Paths supplied to logging options are interpreted by the host-engine side of the request.

Examples

Readiness and failed-job triage

Discover the local IDs, then run the quick suite on one idle GPU before admitting a workload:

```
$ dcgmi discovery --list
$ dcgmi diag --run 1 --entity-id gpu:0
```

If a workload later fails and GPU 0 is removed from service, the medium suite adds memory and PCIe tests. JSON is preferable when another program consumes the result, subject to the release-specific

schema limitation above:

```
$ dcgmi diag --run 2 --entity-id gpu:0 --json
```

When the symptom points to one subsystem, a canonical test name avoids running an entire suite:

```
$ dcgmi diag --run pcie --entity-id gpu:0 --iterations 3
```

Parameter overrides

Override one documented plugin parameter:

```
$ dcgmi diag --run targeted_power \  
  --parameters targeted_power.target_power=300.0
```

Structured output

Request JSON for a named PCIe run:

```
$ dcgmi diag --run pcie --entity-id gpu:0 --json
```

Configuration File

The diagnostic `dcgm-diag-v1` YAML configuration supplies device-specific test permissions, thresholds, and parameter values. NVIDIA's packaged `diag-skus.yaml` provides product defaults; `--configfile` supplies an overlay, and command-line `--parameters` values have final priority for the request.

See [the diagnostic configuration file reference](#) for the complete schema, PCI device and subsystem matching, merge behavior, source precedence, and the fields dispatched to each plugin. A configuration can allow or disable an available plugin and set SKU-specific expectations; it cannot make an absent plugin or unsupported test available.

Example configuration

The following `custom-diag-tests.yaml` overlay enables the memory test for an H100 80GB PCIe GPU and limits the test allocation target to 50 percent of framebuffer memory. It deliberately does not define product-specific performance or power thresholds; use it for the named memory test rather than as a complete numbered-suite profile:

```
skus:
- name: H100 80GB PCIe
  id: '2331'
  memory:
    is_allowed: true
    minimum_allocation_percentage: 50.0
    ll_is_allowed: false
```

Use the device ID for the target GPU rather than applying the H100 identifier to another product. See [GPU Memory Plugin](#) for the memory plugin's parameter and configuration-dispatch contract.

Run the configured memory test on GPU 0:

```
$ dcgmi diag --run memory --entity-id gpu:0 \
  --configfile custom-diag-tests.yaml
```

Further Reading

- [dcgmi diag](#) is the exhaustive command reference.
- [Diagnostic Plugins](#) contains detailed per-plugin references.
- [Diagnostic Configuration File](#) specifies the diagnostic YAML schema, matching, merge behavior, and precedence.
- [Health Monitoring](#) explains passive monitoring during workloads.
- [dcgmi mndiag](#) covers coordinated multi-node diagnostics; it is a separate module and workflow.
- [Debugging and Troubleshooting](#) describes evidence to retain when a diagnostic or workload fails.



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.